Pruning and Distilling a Mixture-of-Experts Language Model for TypeScript Specialization

Sartaj Bhuvaji [1]

Abstract

We apply the Minitron structured pruning and knowledge distillation recipe (Muralidharan et al., 2024; Sreenivas et al., 2024) to a Mixture-of-Experts (MoE) language model, Qwen3-30B-A3B-Base, and specialize the result toward TypeScript. Both Minitron papers prune only dense transformers; we extend their forward-pass-only, activation-based importance-estimation philosophy to a pruning axis neither paper covers – expert pruning. A gate-weighted expert-output-norm criterion (independently converged on by REAP (Lasby et al., 2025) and NVIDIA's Minitron-MoE extension (NVIDIA, 2024)) ranks previously-reported "Super Experts" (Su et al., 2025) first in their respective layers, and a cheap, single-forward-pass depth-pruning proxy (Block Importance) correlates strongly with the expensive, perplexity-based criterion from the original Minitron paper (Pearson $r = 0.91$; both agree on the three most critical layers). We prune, not just score, on all three axes. A depth sweep (nine layer counts, four selection strategies) shows principled selection beating random by up to 9× in relative loss once more than a handful of layers are pruned, and finds contiguous-block removal – the second Minitron paper's preferred depth-pruning approach – a mixed result on this model. An expert-pruning sweep (eight per-layer counts) shows the same pattern at a narrower margin (2–5×), and finds that pruning only the three known Super Experts costs more zero-shot loss than greedy-pruning up to 1,536 ordinary experts, reproducing a prior regression test directly in this pipeline's own metric. A head-pruning sweep (all 7 valid per-group counts under this model's grouped-query attention) shows a much narrower greedy-vs-random margin still (1.2–2.9×), and comparing all three axes at matched pruned-fractions shows experts are by far the most redundant axis on this model, with heads and depth comparably (un)forgiving to prune. This is a living document, updated as further pruning, distillation, and retraining experiments complete; no axis has been combined with another, and no retraining has occurred.

1. Introduction

Minitron (Muralidharan et al., 2024) demonstrated that a large language model can be compressed into a family of smaller models via structured pruning followed by knowledge distillation, using up to 40× fewer training tokens than training each size from scratch while matching or exceeding the accuracy of from-scratch baselines. A follow-up paper (Sreenivas et al., 2024) applied the same recipe under a more realistic constraint – no access to the original pretraining data – and introduced teacher correction and a downstream-task-based depth-pruning criterion.

Both papers prune exclusively dense transformers. Every weight in a dense model's MLP is applied to every token that passes through it. This is not true of a Mixture-of-Experts (MoE) model: an MoE layer routes each token to a small subset of experts (8 of 128, in the model we study here), so most of the model's parameters are never touched by any given token. This work asks what happens when the Minitron recipe is applied to a model with this property, and in particular, how to score the importance of an axis – expert pruning – that has no precedent in either source paper.

Our target model is Qwen3-30B-A3B-Base (Qwen Team, 2025): 48 decoder layers, 32 query / 4 key-value attention heads (GQA), 128 routed experts per layer with 8 active per token, no shared expert, approxi-

[1]Seattle University, Seattle, WA, USA. Correspondence to: Sartaj Bhuvaji <sbhuvaji@seattleu.edu>.

Proceedings of the 42nd International Conference on Machine Learning, Vancouver, Canada. PMLR 267, 2025. Copyright 2025 by the author(s).

mately 30.5B total parameters of which approximately 3.35B are active for any given token. We specialize this model toward TypeScript, using bigcode/the-stack-dedup's TypeScript split (Kocetkov et al., 2022) for both calibration and retraining data, drawn from disjoint parquet shards so the two pools never overlap.

This paper is written incrementally, alongside the project's own commit history: each section is updated with real, measured results only, once the corresponding experiment has actually run. As of this writing, all three pruning axes – depth (Section 6), experts (Section 7), and attention heads (Section 8) – have each been characterized in isolation, but none has been combined with another, retrained, or finalized to one candidate.

2. Related Work

Minitron. Muralidharan et al. (2024) prune Nemotron-4 15B to 8B and 4B using purely activation-based importance estimation (no gradients), scoring attention heads, MLP neurons, and embedding channels via a shared formula that aggregates activations over a small (1024-sample) calibration set, and scoring depth separately via perplexity delta or Block Importance (cosine distance between a layer's input and output). Sreenivas et al. (2024) extend this to settings without the original pretraining data, introducing teacher correction (lightly fine-tuning the teacher on the actual distillation data before pruning) and a downstream-task-based depth-pruning criterion using contiguous-block removal, evaluated on Llama 3.1 8B and Mistral NeMo 12B.

Expert pruning in MoE models. Neither Minitron paper addresses Mixture-of-Experts models. REAP (Lasby et al., 2025) scores expert importance as a gate-weighted expert-output norm, averaged over each expert's routed tokens, and evaluates on Qwen3-30B-A3B, the same architecture family we study. NVIDIA's Model-Optimizer library (NVIDIA, 2024) independently implements the same formula as part of its own Minitron-MoE extension, used to compress Nemotron-3-Nano-30B-A3B. Prior work on Mixtral-scale models found selection-frequency-based pruning performs worst among the criteria it tested (Lu et al., 2024), consistent with our choice not to rely on selection frequency (Section 5.2).

Super Experts. Su et al. (2025) report that Qwen3-30B-A3B contains a small number of experts (three in the first four layers of the model, by their numbering) whose rare but extremely large-magnitude outputs anchor attention sinks; ablating them alone raises WikiText-2 perplexity from 8.70 to 59.86. We treat this as a regression test for our own expert importance metric (Section 5.2).

Table 1. Baseline evaluation of Qwen3-30B-A3B-Base, unpruned.

Task	Setting	Score
HumanEval-TS	pass@1	0.6708
MBPP-TS	pass@1	0.7333
MMLU	5-shot, acc	0.8101
ARC-Challenge	25-shot, acc	0.6536
ARC-Challenge	25-shot, acc_norm	0.6749
Winogrande	5-shot, acc	0.7798

3. Target Model and Data

3.1. Model

Qwen3-30B-A3B-Base (Qwen Team, 2025) is a base (not instruction-tuned) causal language model with 48 Qwen3MoeDecoderLayer blocks, hidden size 2048, 32 query heads and 4 key-value heads (grouped-query attention, head_dim 128), and 128 routed experts per layer (Qwen3MoeExperts, moe_intermediate_size 768) with the top 8 active per token, no shared expert. Every layer is sparse (decoder_sparse_step=1, mlp_only_layers empty). Experts within a layer are not separate submodules but two batched parameter tensors (gate_up_proj, down_proj), with the expert index as the leading dimension – pruning an expert means slicing that dimension.

3.2. Data

Calibration and retraining data are both drawn from bigcode/the-stack-dedup's data/typescript split (71 parquet shards). The first shard is reserved for calibration, the remaining 70 for retraining; because the split is shard-level rather than row-level, the two pools are disjoint by construction, which we additionally verify empirically against each sample's hexsha.

4. Baseline Evaluation of the Unpruned Teacher

Before any pruning decision, we establish the unpruned model's performance on the evaluation suite every subsequent pruning candidate will be measured against. Following both Minitron papers' evaluation settings (temperature 0.2, nucleus sampling top-p = 0.95 for generation tasks), we report:

HumanEval-TS and MBPP-TS (both MultiPL-E TypeScript translations) are treated as the primary,

gating tier, since they measure the capability this project specializes toward. MMLU, ARC-Challenge, and Winogrande are tracked but not optimized for, giving a comparison point against both Minitron papers' own general-knowledge tables without over-indexing on general capability at the expense of TypeScript performance.

5. Importance Estimation

Both Minitron papers score importance from forward-pass activations only, never gradients, over a small calibration set. We follow the same philosophy for every axis, including the one neither paper covers.

5.1. Attention Heads

Head importance uses the original formula unchanged. For head h in a given layer, let $a_{t,h} \in \mathbb{R}^{d_{\text{head}}}$ be that head's output for calibration token t, prior to the W^O merge. The per-token score is its L_2 norm, aggregated over every token t in the calibration set:

$$F_h = \text{agg}_t \left(\|a_{t,h}\|_2 \right), \tag{1}$$

where agg is a pluggable aggregation function – Muralidharan et al. (2024) ablate mean-of-absolute-value, L_2 norm, and variance, finding the best choice is not the same for every axis; we use the L_2 variant throughout this paper. Across all 48 layers (48 calibration samples, mean length 177 tokens), scores range from 2.39 to 967.72, with the single largest value at the model's final layer (layer 47) – visible as the dark band at the bottom of Figure 1.

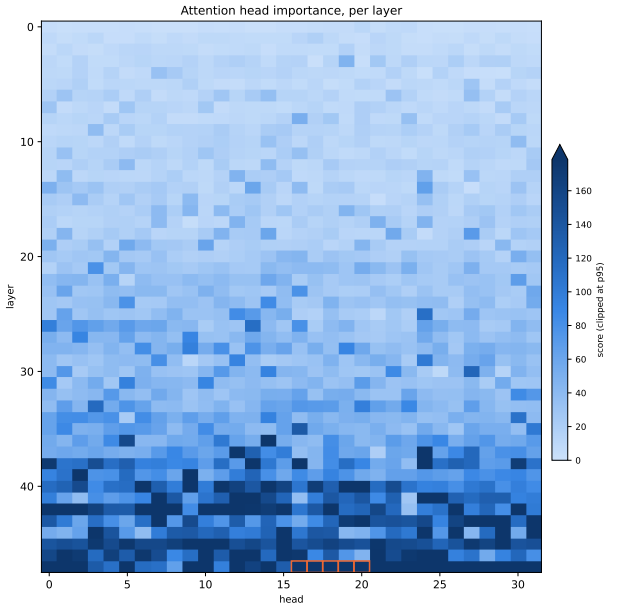


Figure 1. Head importance (F_h, Equation (1)), all 48 layers × 32 heads. Color is clipped at the 95th percentile (outlined cells mark the true top 5) since a linear scale to the true maximum leaves the rest of the grid indistinguishable from zero.

5.2. Experts

Neither Minitron paper offers a formula for this axis. We adopt

$$S_j = \text{mean}_{x \in X_j} \left[g_j(x) \cdot \|f_j(x)\|_2 \right], \tag{2}$$

where X_j is the set of tokens routed to expert j, $g_j(x)$ is that token's renormalized routing weight, and $f_j(x)$ is the expert's output – the formula independently reached by both REAP (Lasby et al., 2025) and NVIDIA Model-Optimizer's (NVIDIA, 2024) Minitron-MoE extension, in preference to two alternatives (selection frequency alone; router logit magnitude alone) both cited sources report underperforming it (see also Section 2's note on selection-frequency-based pruning more broadly).

Unlike head importance, there is no dense batch/sequence grid to aggregate over: routing is sparse and data-dependent per token, so S_j is computed from a flat list of (token, expert) assignments rather than a fixed-shape tensor, mirroring how the model's own expert-dispatch loop iterates (Algorithm 1).

An expert with zero routed tokens in the calibration set scores 0 and ranks lowest by construction – appropriate for a calibration-set-specific score, but a real risk if a rare-but-critical expert happens to be undersampled by chance, which is exactly what the Super Experts check below guards against.

As a validation check, not a designed experiment: all three Super Experts reported by Su et al. (2025) for this exact model (layer 1 / expert 68, layer 2 / expert 92, layer 3 / expert 82) rank first among all 128 experts in their respective layers under this metric, on real calibration data, at two different sample sizes (24 and 48 samples) – the three isolated bright cells in Figure 2. On average across all 48 layers, the top ∼16% of experts (20 of 128) already account for 33% of a layer's total importance mass (Figure 3), indicating substantial redundancy among the remaining experts as a candidate for pruning.

5.3. Depth: Block Importance and Perplexity

We implement both depth-scoring criteria from Muralidharan et al. (2024). Block Importance (BI) for

Algorithm 1 Expert importance from routing data

Input: expert indices $e_1, \ldots, e_N$, gate weights $g_1, \ldots, g_N$, expert outputs $f_1, \ldots, f_N \in \mathbb{R}^d$ for every (token, selected-expert) pair in the calibration set; number of experts E
sum[1..E] $\leftarrow 0$; count[1..E] $\leftarrow 0$
for $i = 1$ to N do
  sum[e_i] $\leftarrow$ sum[e_i] $+ g_i \cdot \|f_i\|_2$
  count[e_i] $\leftarrow$ count[e_i] $+ 1$
end for
for $j = 1$ to E do
  $S_j \leftarrow$ count[j] > 0 ? sum[j]/count[j] : 0
end for
Return: $S_1, \ldots, S_E$

Algorithm 2 Perplexity-delta depth scoring

Input: model θ with layers $1, \ldots, L$; calibration batches $\mathcal{D}$
base $\leftarrow$ MeanLoss($\theta, \mathcal{D}$)
for $\ell = 1$ to L do
  replace layer ℓ's forward with the identity function
  $\Delta_\ell \leftarrow$ MeanLoss($\theta, \mathcal{D}$) $-$ base
  restore layer ℓ's original forward function
end for
Return: $\Delta_1, \ldots, \Delta_L$

layer ℓ, with input activations X_ℓ and output activations $X_{\ell+1}$ over calibration tokens t, is

$$\mathrm{BI}_\ell = 1 - \mathbb{E}_t \left[\frac{X_{\ell,t}^\top X_{\ell+1,t}}{\|X_{\ell,t}\|_2 \|X_{\ell+1,t}\|_2} \right], \tag{3}$$

computable in the same forward pass that produces the two activations. The perplexity-delta criterion instead removes one layer at a time and measures the resulting change in held-out loss relative to the unmodified baseline:

$$\Delta_\ell = \mathcal{L}(\theta \setminus \ell) - \mathcal{L}(\theta), \tag{4}$$

where $\mathcal{L}$ is mean cross-entropy loss over the calibration set and $\theta \setminus \ell$ denotes the model with layer ℓ temporarily skipped (Algorithm 2). This costs $L+1$ forward passes for an L-layer model, versus BI's one, and is run on a smaller calibration sample (16 samples, vs. 48 for BI) for that reason.

Both criteria independently identify the same pattern: the first and last layers of the model matter far more than the middle two-thirds (Figure 4). BI's highest score is at layer 0 (0.488) and second-highest at layer 47 (0.415); its lowest is layer 12 (0.011). The perplexity criterion shows the same edges-matter-most shape at a far more extreme scale – layer 0's removal costs 1.99 nats of cross-entropy, next-highest is layer 47 at 0.60 nats, and the lowest (layer 14) costs only -0.0003 nats.

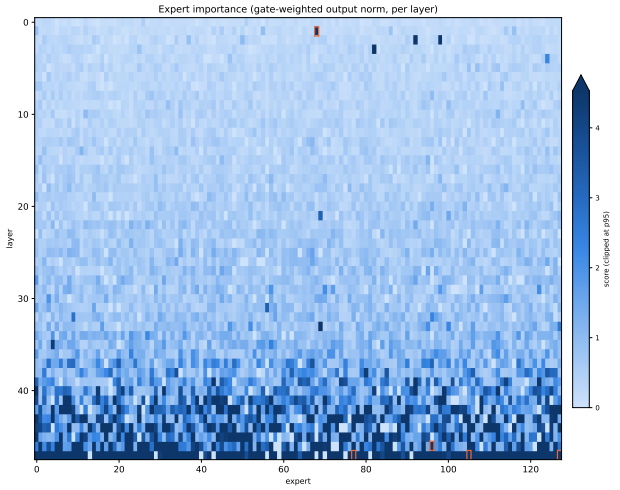


Figure 2. Expert importance (S_j), all 48 layers $\times$ 128 experts, same clipping convention as Figure 1 (outlined cells mark the true top 5 model-wide, not per-layer). Only one Super Expert (Su et al., 2025) (layer 1, expert 68) is among them; the other two rank highest only within their own layer, not model-wide – but the 95th percentile color clip still renders all three as visually saturated, since each is well above that threshold.

Across all 48 layers, the Pearson correlation between BI and the perplexity delta is $r = 0.91$ (Figure 5), and the two criteria pick the identical top-3 most critical layers (0, 47, 1). Given the 49$\times$ cost difference between the two, BI is a reliable cheap stand-in for perplexity-based scoring when iterating over many candidates, reserving the expensive criterion for confirming a final choice.

Figure 6 places the three cheap, single-forward-pass methods – heads, experts, BI – on the same (min-max normalized) axis; Figure 5 above already gives the fourth, perplexity, directly against BI. Heads and experts agree closely with each other throughout the model; BI diverges from both through the middle third, agreeing with them only at the two edges – the same edges where Figure 5 shows it also agrees with perplexity.

6. Depth Pruning: Zero-Shot Loss vs. Layers Pruned

Sections 5.2 and 5.3 only score which components matter; this section actually prunes them. bonsai.pruning.depth.prune_layers permanently removes a chosen set of decoder layers – unlike skip_layer's

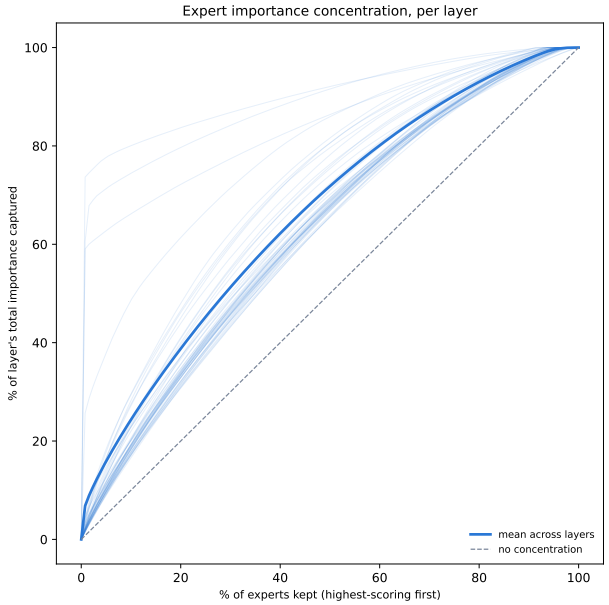


Figure 3. Per-layer cumulative importance captured by the top-k highest-scoring experts (thin lines: all 48 layers; bold: mean). The dashed diagonal is the no-concentration reference.

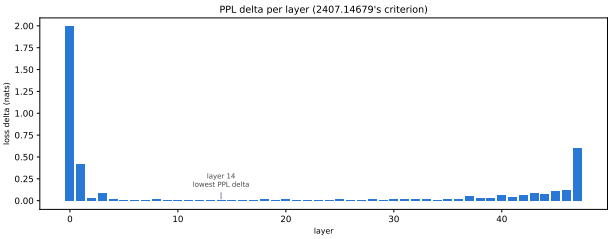


Figure 4. Perplexity delta Δ_ℓ (Equation (4)) per layer.

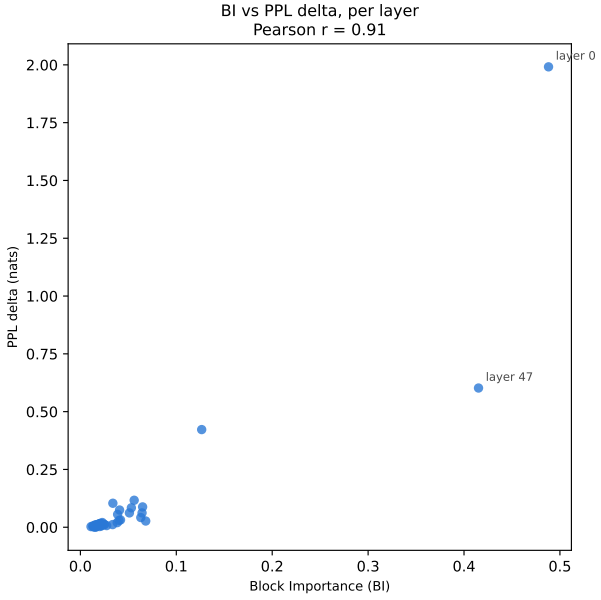


Figure 5. BI (Equation (3)) vs. perplexity delta (Equation (4)), one point per layer.

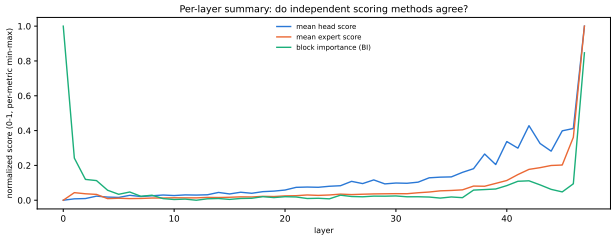


Figure 6. Mean head score, mean expert score, and BI, per layer, min-max normalized within each method.

temporary identity-swap used for scoring – and renumbers each surviving layer's attention-cache index to match its new position, since the default KV cache indexes by position and breaks on the non-contiguous indices removal otherwise leaves behind. bonsai.importance.depth.mean_loss measures the resulting model's zero-shot loss, here on 32 fresh calibration samples (seed 1, a disjoint draw from every prior calibration slice in this paper).

We sweep nine layer counts ($n \in \{1, 2, 4, 6, 8, 12, 16, 20, 24\}$ of 48) under four selection strategies: bi_greedy (the n lowest-BI layers, Equation (3)); ppl_greedy (the n lowest-perplexity-delta layers, Equation (4)); contiguous_bi (the single contiguous block of n layers with the lowest summed BI – testing Sreenivas et al. (2024)'s own claim, citing Gromov et al., that contiguous removal beats scattered removal); and random (n random layers, 3 seeds), the control every principled strategy has to beat. Each candidate costs one forward pass, since the ranking is already decided.

Principled selection clearly beats random at every count tested. In relative-loss-increase terms (Table 2), the gap approaches an order of magnitude as n grows: at $n = 16$, bi_greedy is +41.6% over baseline versus random's +372.6% mean (9.0×), and at $n = 24$, bi_greedy/ppl_greedy are +153.8%/+135.6% against random's +1000.1% mean (6.5–7.4×; in raw loss, 2.45/2.28 against a range of 4.81–14.65 across the three seeds). bi_greedy and ppl_greedy track each other closely at every count – not merely in the ranking correlation already established ($r = 0.91$, Section 5.3) but in the actual pruning outcome – reinforcing BI as the working criterion for iterating on candidates.



Figure 7. Zero-shot loss vs. layers pruned. Random shown as its 3-seed mean (dashed) and min-max band. Log-scale y-axis.



Figure 8. Zero-shot loss vs. experts pruned per layer. Random shown as its 3-seed mean (dashed) and min-max band; the horizontal reference line is the super_experts_only candidate. Log-scale y-axis.

Table 2. Loss increase over baseline (0.9670), by strategy and layer count.

n	bi_gr.	ppl_gr.	contig.	rand. (mean)
1	+0.3%	+0.9%	+0.3%	+5.3%
2	+1.8%	+1.4%	+0.9%	+7.5%
4	+4.3%	+3.8%	+3.6%	+14.5%
6	+8.1%	+7.0%	+7.3%	+31.7%
8	+12.0%	+11.9%	+11.3%	+40.7%
12	+21.9%	+22.2%	+24.6%	+101.8%
16	+41.6%	+46.4%	+44.5%	+372.6%
20	+76.7%	+73.5%	+76.2%	+568.8%
24	+153.8%	+135.6%	+211.0%	+1000.1%

Contiguous-block removal is a genuinely mixed result on this model, not a clean win either way. It is competitive – marginally best – at small n ($n = 2$: +0.9% vs. bi_greedy's +1.8%), but falls behind both greedy strategies as n grows, most visibly by $n = 24$ (+211.0% vs. +135.6–153.8%). A plausible reading: BI's lowest-scoring layers already cluster in a contiguous run in the middle of the model (Figure 6), so for small n the two approaches pick nearly the same layers anyway; forcing strict contiguity only starts costing something once n exceeds what that natural cluster contains, pushing the contiguous search into a wider, less-optimal window.

7. Expert Pruning: Zero-Shot Loss vs. Experts Pruned

Section 6 pruned layers; this section prunes experts. bonsai.pruning.experts.prune_experts permanently slices the chosen experts out of Qwen3MoeExperts's gate_up_proj and down_proj and the matching rows of Qwen3MoeTopKRouter's weight matrix, per layer. Unlike the layer-index renumbering Section 6 needed, no such step is required here: the model re-derives which experts were hit fresh from routing indices on every forward pass rather than depending on a fixed per-expert identity, so a pruned expert's neighbors need no bookkeeping to stay consistent. We measure the resulting zero-shot loss the same way as Section 6, on the same 32 fresh calibration samples.

We sweep eight per-layer expert counts ($n \in \{8, 16, 32, 48, 64, 80, 96, 112\}$ of 128), pruned uniformly across all 48 sparse layers, under two selection strategies: greedy (the n lowest-S_j experts per layer, Equation (2), with the three Super Experts identified in Section 5.2 exempted from the ranking entirely rather than merely checked afterward); and random (n random experts per layer, same exemption, 3 seeds). We additionally evaluate one candidate outside the count sweep – super_experts_only – that prunes just the three known Super Experts and nothing else, to reproduce Su et al. (2025)'s own regression test directly in this pipeline's loss metric rather than only by citation.

Greedy selection beats random at every count tested (Table 3), though by a narrower margin than depth pruning's: roughly 2–5× in relative-loss terms here, versus close to an order of magnitude for layers (Section 6). A plausible reading is that 128 experts per layer offers far more substitutable choices than 48 layers does, so principled selection matters less dramatically even though it still matters clearly. Pruning is close to free up to a point: $n = 8$ (384 experts total, across all layers) lands at the same loss as the unpruned baseline (0.9670); the curve only bends noticeably past $n = 48$–64.

The starkest finding in this sweep is not on the main curve at all: pruning only the three known Super Experts – 3 of the model's 6,144 experts – costs more zero-shot loss (+51.8%) than greedy-pruning up to

Table 3. Loss increase over baseline (0.9670), by strategy and per-layer expert count. "total" is experts removed across all 48 layers.

n	total	greedy	random (mean)
8	384	0.0%	+7.7%
16	768	+3.0%	+19.0%
32	1536	+9.6%	+43.0%
48	2304	+13.7%	+71.8%
64	3072	+25.5%	+105.9%
80	3840	+46.3%	+167.4%
96	4608	+83.2%	+256.4%
112	5376	+229.7%	+434.0%

1,536 ordinary experts ($n = 32$, +9.6%). Greedy selection does not cross the super_experts_only line until somewhere between $n = 80$ and $n = 96$ (3,840–4,608 experts), roughly 1,300–1,500× as many experts as the three Super Experts alone. This reproduces Su et al. (2025)'s claim (isolated ablation of these experts alone raised WikiText-2 perplexity from 8.70 to 59.86) directly in this pipeline's own loss metric and on this project's own TypeScript calibration data, not merely by citation, and is the concrete justification for exempting these experts from the ranking (Section 5.2) rather than relying only on the score itself to never rank them low.

8. Head Pruning: Zero-Shot Loss vs. Query Heads Pruned

Section 7 pruned experts; this section prunes attention heads. bonsai.pruning.heads.prune_heads permanently slices the chosen query heads out of Qwen3MoeAttention's q_proj (output rows) and o_proj (input columns) at head boundaries. Unlike the other two axes, heads cannot be selected freely: Qwen3-30B-A3B-Base's grouped-query attention (GQA) shares every 8 consecutive query heads onto one key/value head (num_key_value_groups = 8, Section 3), and Qwen3MoeAttention.forward reads that value fresh off the module on every call via repeat_kv – pruning a ragged count per group would leave it undefined. We therefore prune the same count of query heads from every key/value group in a layer, never an entire group (which would orphan its key/value head), and recompute self_attn.num_key_value_groups for the new count. We measure the resulting zero-shot loss the same way as Sections 6 and 7, on the same 32 fresh calibration samples.

This constraint fixes the sweep's entire valid search space rather than leaving a larger one to sample from, unlike the other two axes: group size 8 admits exactly 7 per-group counts ($k \in \{1, \ldots, 7\}$, since $k = 8$ would orphan a key/value head), so we sweep all of them under two selection strategies: greedy (the k lowest-F_h-scoring query heads per group, Equation (1)); random (k random heads per group, 3 seeds). This run used a rented 4x A6000 instance via device_map="auto" sharding rather than the single 80GB GPU the other two sweeps used, since no H100/A100-80GB capacity was available at launch time.

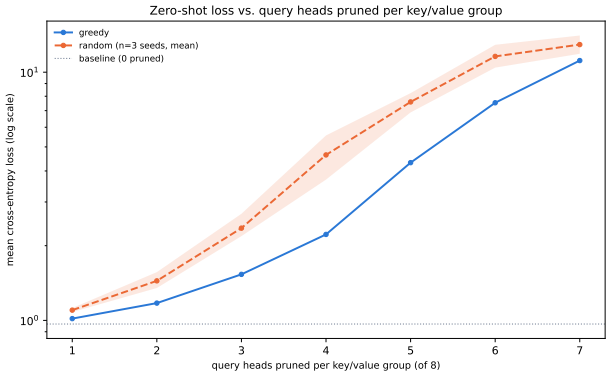


Figure 9. Zero-shot loss vs. query heads pruned per key/value group. Random shown as its 3-seed mean (dashed) and min-max band. Log-scale y-axis.

Table 4. Loss increase over baseline (0.9668), by strategy and per-group head count. "total" is query heads removed across all 48 layers.

k	total	greedy	random (mean)
1	192	+5.2%	+13.8%
2	384	+21.4%	+49.3%
3	576	+58.5%	+143.4%
4	768	+129.6%	+380.0%
5	960	+346.8%	+684.9%
6	1152	+678.9%	+1097.5%
7	1344	+1052.1%	+1236.4%

Greedy selection beats random at every count tested (Table 4), but by a much narrower margin than either other axis, and one that shrinks as k grows: 2.7× at $k = 1$ down to 1.2× at $k = 7$, versus depth's up to 9× (Section 6) and experts' 2–5× (Section 7).

The narrowing greedy/random gap raises an obvious question: is this model's attention-head axis simply more redundant than depth or experts, so any reasonable subset survives about as well as any other – or is it the least redundant, unable to absorb pruning gracefully regardless of which heads are chosen? Comparing absolute cost at every pruned-fraction this sweep shares with the other two answers it. Depth's counts coincide with heads' at 3 fractions (12.5%, 25%, 50%); experts' counts coincide at all 7. At every shared point, expert pruning is dramatically cheaper

than either heads or depth – e.g. at 50% pruned, greedy costs +25.5% for experts versus +129.6% for heads and +153.8% for depth. Heads and depth track each other closely at all 3 points they share (12.5%: +5.2% vs. +8.1%; 25%: +21.4% vs. +21.9%; 50%: +129.6% vs. +153.8%), with depth's greedy loss slightly higher every time – a small, consistent difference on a single-seed, 32-sample run, not one we lean on as a strong ranking claim. The robust reading is that experts are far more redundant than either heads or depth on this model; heads and depth look comparably (un)forgiving to prune, and the narrowing greedy/random gap on heads reflects that low redundancy – with few good substitutes available in a group of 8, even a bad (random) choice is not that much worse than a good one, not because head choice stops mattering.

9. Discussion and Open Questions

Several design decisions remain open and are logged here rather than resolved prematurely:

- A specific target layer count. Section 6 characterizes the cost curve but does not commit to one candidate to carry into later axis-combination and retraining – that choice depends on a compute/accuracy tradeoff not yet made.
- Downstream-depth-pruning proxy. Sreenivas et al. (2024)'s preferred depth criterion ranks contiguous layer blocks by downstream-task accuracy after removal (Winogrande, in their case), not by BI or perplexity. No TypeScript equivalent of Winogrande exists; HumanEval-TS or MBPP-TS pass-rate on partially-pruned models is the direct analogue but expensive per candidate block, and a cheaper token-level-accuracy proxy is under consideration.
- A specific target per-layer expert count. Section 7 characterizes the cost curve up to $n = 112$ of 128 but does not commit to one candidate to carry into later axis-combination and retraining, for the same reason as the layer-count question above.
- Uniform vs. per-layer expert counts. Section 7's sweep uses uniform expert counts per layer throughout, as a simplifying starting choice rather than one settled by confirmed evidence: scoping research surfaced conflicting claims on this exact question (one evolutionary-search paper reports the opposite – non-uniform allocation beating uniform by up to 19.6% on MATH-500 at matched sparsity), and neither claim has been independently re-verified by this project. Whether a per-layer search would do meaningfully better than uniform on this model remains untested here directly.
- A specific target per-group head count. Section 8 characterizes the cost curve across all 7 valid per-key/value-group counts but does not commit to one candidate, for the same reason as the two target-count questions above.
- Free (non-uniform-per-group) head selection. Section 8's GQA constraint requires the same count pruned from every key/value group in a layer; whether a free per-head search across group boundaries would do meaningfully better is untested, since prune_heads does not support it by design.
- Interaction between axes. Muralidharan et al. (2024) find that depth and width importance rankings can flip after retraining relative to their zero-shot rankings; whether the same holds when experts are combined with depth and width has no precedent to draw on. All three axes are now characterized in isolation (Sections 6 to 8), so this is the next open question this project is positioned to answer empirically, not a hypothetical for later.
- Lightweight-retrain token budget. Muralidharan et al. (2024)'s 1.8B-token budget was calibrated for a 15B dense model, not a ∼3.35B active-parameter MoE; the right budget for this model's post-combination retraining passes (the item above) has no direct precedent to scale from.

No results are reported for these open questions; they are listed to make the project's actual state legible, not to imply an answer.

Acknowledgements

This is an independent research project with no external funding.

References

Kocetkov, D., Li, R., Allal, L. B., et al. The stack: 3 tb of permissively licensed source code. https://huggingface.co/datasets/bigcode/the-stack-dedup, 2022.

Lasby, M., Lazarevich, I., Sinnadurai, N., Lie, S., Ioannou, Y., and Thangarasa, V. Reap: Router-weighted expert activation pruning for mixture-

of-experts language models. arXiv preprint arXiv:2510.13999, 2025.

Lu, X., Liu, Q., Xu, Y., Zhou, A., Huang, S., Zhang, B., Yan, J., and Li, H. Not all experts are equal: Efficient expert pruning and skipping for mixture-of-experts large language models. arXiv preprint arXiv:2402.14800, 2024.

Muralidharan, S., Sreenivas, S. T., Joshi, R., Chochowski, M., Patwary, M., Shoeybi, M., Catanzaro, B., Kautz, J., and Molchanov, P. Compact language models via pruning and knowledge distillation. arXiv preprint arXiv:2407.14679, 2024.

NVIDIA. Nvidia model optimizer. https://github.com/NVIDIA/Model-Optimizer, 2024.

Qwen Team. Qwen3 technical report. arXiv preprint arXiv:2505.09388, 2025. Model weights: https://huggingface.co/Qwen/Qwen3-30B-A3B-Base.

Sreenivas, S. T., Muralidharan, S., Joshi, R., Chochowski, M., Patwary, M., Shoeybi, M., Catanzaro, B., Kautz, J., and Molchanov, P. Llm pruning and distillation in practice: The minitron approach. arXiv preprint arXiv:2408.11796, 2024.

Su, Z., Li, Q., Zhang, H., Ye, W., Xue, Q., Qian, Y., Xie, Y., Wong, N., and Yuan, K. Unveiling super experts in mixture-of-experts large language models. arXiv preprint arXiv:2507.23279, 2025.